文件系统构建

第一步：环境搭建与 SDK 获取

这是所有工作的基础，确保环境正确能避免后续很多问题。

· 获取 SDK 源码：`atk_dlrk3506_linux6.1_release_v1.3.1_20260326`

· 解压与修复：

1. 将 SDK 源码包拷贝到 Ubuntu 系统中，并解压。

2. 进入解压后的 SDK 根目录，执行 ./repo.sh 命令。

· 安装依赖包：在终端中执行以下命令，安装编译所需的工具和库：

```
sudo apt-get update
sudo apt-get install repo git ssh make gcc libssl-dev liblz4-tool expect
g++ patchelf chrpath gawk texinfo diffstat binfmt-support qemu-user-static
live-build bison flex fakeroot cmake gcc-multilib g++-multilib unzip
device-tree-compiler ncurses-dev
```

第二步：系统配置与编译

环境就绪后，开始配置和编译你的专属系统。

1. 加载板卡配置：

在 SDK 根目录执行以下命令，加载预设的板卡配置：

```
./build.sh -h
```

在这里可以看到 `build` 脚本的使用说明

```
fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR: ~/atk_dlrk3506_linux6...

1. 01_atk_dlrk3506b_mipi720x1280_nand_ubi_squashfs_defconfig
2. 02_atk_dlrk3506b_mipi720x1280_nand_ubi_squashfs_thunderboot_defconfig
3. 03_atk_dlrk3506b_mipi720x1280_nand_ubi_ubifs_amp_linux_defconfig
4. 04_atk_dlrk3506b_mipi720x1280_nand_ubi_ubifs_amp_mcu_defconfig
5. 05_atk_dlrk3506b_mipi720x1280_nand_ubi_ubifs_defconfig
6. 06_atk_dlrk3506b_rgb1024x600_nand_ubi_ubifs_defconfig
7. 07_atk_dlrk3506b_mipi720x1280_nand_ubi_ubifs_thunderboot_defconfig
8. 08_atk_dlrk3506b_mipi720x1280_sd_defconfig
9. 09_atk_dlrk3506j_mipi720x1280_emmc_amp_linux_defconfig
10. 10_atk_dlrk3506j_mipi720x1280_emmc_amp_mcu_defconfig
11. 11_atk_dlrk3506j_mipi720x1280_emmc_defconfig
12. 12_atk_dlrk3506j_mipi720x1280_emmc_thunderboot_defconfig
13. 13_atk_dlrk3506j_mipi720x1280_nand_ubi_squashfs_defconfig
14. 14_atk_dlrk3506j_mipi720x1280_nand_ubi_squashfs_thunderboot_defconfig
15. 15_atk_dlrk3506j_mipi720x1280_nand_ubi_ubifs_amp_linux_defconfig
16. 16_atk_dlrk3506j_mipi720x1280_nand_ubi_ubifs_amp_mcu_defconfig
17. 17_atk_dlrk3506j_mipi720x1280_nand_ubi_ubifs_defconfig
18. 18_atk_dlrk3506j_mipi720x1280_nand_ubi_ubifs_thunderboot_defconfig
19. 19_atk_dlrk3506b_automipi_nand_ubi_squashfs_defconfig
20. 20_atk_dlrk3506b_automipi_sd_defconfig
21. 21_atk_dlrk3506j_automipi_nand_ubi_squashfs_defconfig
22. 22_atk_dlrk3506j_automipi_emmc_defconfig
23. 23_atk_dlrk3506b_mipi800x1280_nand_ubi_squashfs_defconfig
```

选择板级文件：

```
./build.sh lunch
```

```
fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR: ~/atk_dlrk3506_linux6...
2. 02_atk_dlrk3506b_mipi720x1280_nand_ubi_squashfs_thunderboot_defconfig
3. 03_atk_dlrk3506b_mipi720x1280_nand_ubi_ubifs_amp_linux_defconfig
4. 04_atk_dlrk3506b_mipi720x1280_nand_ubi_ubifs_amp_mcu_defconfig
5. 05_atk_dlrk3506b_mipi720x1280_nand_ubi_ubifs_defconfig
6. 06_atk_dlrk3506b_rgb1024x600_nand_ubi_ubifs_defconfig
7. 07_atk_dlrk3506b_mipi720x1280_nand_ubi_ubifs_thunderboot_defconfig
8. 08_atk_dlrk3506b_mipi720x1280_sd_defconfig
9. 09_atk_dlrk3506j_mipi720x1280_emmc_amp_linux_defconfig
10. 10_atk_dlrk3506j_mipi720x1280_emmc_amp_mcu_defconfig
11. 11_atk_dlrk3506j_mipi720x1280_emmc_defconfig
12. 12_atk_dlrk3506j_mipi720x1280_emmc_thunderboot_defconfig
13. 13_atk_dlrk3506j_mipi720x1280_nand_ubi_squashfs_defconfig
14. 14_atk_dlrk3506j_mipi720x1280_nand_ubi_squashfs_thunderboot_defconfig
15. 15_atk_dlrk3506j_mipi720x1280_nand_ubi_ubifs_amp_linux_defconfig
16. 16_atk_dlrk3506j_mipi720x1280_nand_ubi_ubifs_amp_mcu_defconfig
17. 17_atk_dlrk3506j_mipi720x1280_nand_ubi_ubifs_defconfig
18. 18_atk_dlrk3506j_mipi720x1280_nand_ubi_ubifs_thunderboot_defconfig
19. 19_atk_dlrk3506b_automipi_nand_ubi_squashfs_defconfig
20. 20_atk_dlrk3506b_automipi_sd_defconfig
21. 21_atk_dlrk3506j_automipi_nand_ubi_squashfs_defconfig
22. 22_atk_dlrk3506j_automipi_emmc_defconfig
23. 23_atk_dlrk3506b_mipi800x1280_nand_ubi_squashfs_defconfig
24. 24_atk_dlrk3506b_mipi800x1280_nand_ubi_squashfs_thunderboot_defconfig
25. 25_atk_dlrk3506b_mipi800x1280_nand_ubi_ubifs_amp_linux_defconfig
```

这里我选择的是第 19 个，对应我的系统板 `nandflash` 启动，只读文件系统防止被更改内核

2. 定制 Buildroot：

自定义文件系统，比如添加软件包，可以进入 buildroot/目录，运行 make menuconfig 进行图形化配置。

3. 开始全自动编译：

在 SDK 根目录执行./build.sh 命令。该脚本会自动完成编译 U-Boot、内核、Buildroot 根文件系统，并打包成 update.img 固件的全部工作，约耗时 2 小时，实验结果如下

```
boot    boot.img
rootfs  rootfs.img
userdata        userdata.img
Packing /home/alientek/atk-dlrk3506_linux6.1_release_v1.3.0_20260304/output/firmware/update.img for update...
Android Firmware Package Tool v2.29
------ PACKAGE ------
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0xc5,userspace=0x1
Add file: ./parameter.txt
parameter,Add file: ./parameter.txt done,offset=0x1000,size=0x1d1,userspace=0x1,flash_address=0x00000000
Add file: ./MiniLoaderAll.bin
bootloader,Add file: ./MiniLoaderAll.bin done,offset=0x1800,size=0x421c0,userspace=0x85
Add file: ./uboot.img
uboot,Add file: ./uboot.img done,offset=0x44000,size=0x400000,userspace=0x800,flash_address=0x00002000
Add file: ./misc.img
misc,Add file: ./misc.img done,offset=0x444000,size=0xc000,userspace=0x18,flash_address=0x00004000
Add file: ./recovery.img
recovery,Add file: ./recovery.img done,offset=0x450000,size=0xdb3400,userspace=0x1b67,flash_address=0x00004a00
Add file: ./boot.img
boot,Add file: ./boot.img done,offset=0x1203800,size=0x5e4a00,userspace=0xbca,flash_address=0x0000ba00
Add file: ./rootfs.img
rootfs,Add file: ./rootfs.img done,offset=0x17e8800,size=0x6580000,userspace=0xcb00,flash_address=0x00010a00
Add file: ./userdata.img
userdata,Add file: ./userdata.img done,offset=0x7d68800,size=0xc80000,userspace=0x1900,flash_address=0x0006aa00
Add CRC...
Make firmware OK!
------ OK ------
********rkImageMaker ver 2.29********
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!

Run 'make edit-package-file' if you want to change the package-file.

Running mk-updateimg.sh - build_updateimg succeeded.
Images under /home/alientek/atk-dlrk3506_linux6.1_release_v1.3.0_20260304/output/firmware/ are ready!
Running mk-firmware.sh - build_firmware succeeded.
Running 99-all.sh - build_all succeeded.
```

第三步：烧录镜像

编译完成后，可以在 rockdev/目录下找到 update.img 固件。你可以用以下任一工具将它烧录到开发板：

· 使用 Linux 命令行 (upgrade_tool)：

1. 进入 tools/linux/Linux_Upgrade_Tool/目录,命令行运行./upgrade_tool。

```
fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR: ~/atk_dlrk3506_linux6...
ListDevice:             LD
SwitchDevice:           SD
UpgradeFirmware:        UF <Firmware> [-noreset]
UpgradeLoader:          UL <Loader> [-noreset] [FLASH|EMMC|SPINOR|SPINAND]
DownloadImage:          DI <-p|-b|-k|-s|-r|-m|-u|-t|-re image>
DownloadBoot:           DB <Loader>
EraseFlash:             EF <Loader|firmware>
PartitionList:          PL
WriteSN:                SN <serial number>
ReadSN:                 RSN
ReadComLog:             RCL <File>
CreateGPT:              GPT <Input Parameter> <Output Gpt>
SwitchStorage:          SSD
SwitchUSB3:             SU3
ShowFirmwareInfo:       SFI <Firmware> [entryNo]
ExtractFirmware:        EXF <Firmware|Loader> <Output Dir>
----------------Professional Command -----------------
TestDevice:             TD
ResetDevice:            RD [subcode]
ResetPipe:              RP [pipe]
ReadCapability:         RCB
ReadFlashID:            RID
ReadFlashInfo:          RFI
ReadChipInfo:           RCI
```

2. 连接硬件：用 USB 线连接开发板的 OTG 口和 PC。

3. 进入烧录模式：按住开发板的 BOOT 或 RECOVERY 键，然后上电或按复位键，约 2 秒后松开。

4. 执行烧录命令：

```
sudo ./rkflash.sh
```

或者烧录 update

```
sudo ./rkflash.sh update
```

我使用的是全烧写，结果如下

```
alientek@alientek:~/ATK-DLRK3506$ sudo ./rkflash.sh all
Using /home/alientek/ATK-DLRK3506/tools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool/config.ini
Loading loader...
Loading loader failed,err=-1!
Using /home/alientek/ATK-DLRK3506/tools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool/config.ini
directlba=1,first4access=0,gpt=1,env=0
Write gpt...
Write gpt ok.
Using /home/alientek/ATK-DLRK3506/tools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool/config.ini
directlba=1,first4access=0,gpt=1,env=0
Download uboot start...(0x00002000)
Write file...(100%)
Download image ok.
Using /home/alientek/ATK-DLRK3506/tools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool/config.ini
check download item failed!
Using /home/alientek/ATK-DLRK3506/tools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool/config.ini
directlba=1,first4access=0,gpt=1,env=0
Download boot start...(0x0000ba00)
Write file...(100%)
Download image ok.
Using /home/alientek/ATK-DLRK3506/tools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool/config.ini
directlba=1,first4access=0,gpt=1,env=0
Download recovery start...(0x00004a00)
Write file...(100%)
Download image ok.
Using /home/alientek/ATK-DLRK3506/tools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool/config.ini
directlba=1,first4access=0,gpt=1,env=0
Download misc start...(0x00004000)
Write file...(100%)
Download image ok.
Using /home/alientek/ATK-DLRK3506/tools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool/config.ini
directlba=1,first4access=0,gpt=1,env=0
No found oem in the parameter,please check!
Using /home/alientek/ATK-DLRK3506/tools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool/config.ini
directlba=1,first4access=0,gpt=1,env=0
Download userdata start...(0x00065a00)
Write file...(100%)
Download image ok.
Using /home/alientek/ATK-DLRK3506/tools/linux/Linux_Upgrade_Tool/Linux_Upgrade_Tool/config.ini
directlba=1,first4access=0,gpt=1,env=0
Download rootfs start...(0x00010a00)
Write file...(100%)
```

第四步：验证与后续开发

进入文件系统，输入 ls /可以显示出文件目录如下

```
fanzeny@fanzeny-ASUS-TUF-Gaming-F15-FX507ZR: ~

存储NAND的核心板出厂默认为只读文件系统,只有/user ata分区可读写
若需要将文件系统设置为可读可写,请烧写资料中提供  可读写固件!
若需要修改文件系统路径下的文件,如/etc/下的文件请烧写资料中提供的可读写固件!
实际生产建议设置rootfs为只读,开发可以使用读写固件,生产再使用只读固件
*************************************************************************
root@ATK-DLRK3506:/# /ls
sh: /ls: No such file or directory
root@ATK-DLRK3506:/# ls /
bin                lib                proc               udisk
busybox.fragment   lib32              root               userdata
data               linuxrc            run                usr
dev                media              sbin               var
etc                mnt                sdcard
home               oem                sys
info               opt                tmp
root@ATK-DLRK3506:/# df -h
Filesystem                Size      Used Available Use% Mounted on
/dev/root                85.9M     85.9M         0 100% /
devtmpfs                 95.4M         0     95.4M   0% /dev
tmpfs                    95.4M     80.0K     95.4M   0% /var/log
tmpfs                    95.4M     20.0K     95.4M   0% /tmp
tmpfs                    95.4M    176.0K     95.3M   0% /run
tmpfs                    95.4M         0     95.4M   0% /dev/shm
```

后续的驱动开发可以进入高效的迭代模式：修改 → 编译 → 传输 → 测试。

· 修改：根据需要修改内核源码或设备树（DTS）。

· 编译：使用./build.sh kernel 命令单独编译内核。

· 传输：将编译出的内核模块（.ko 文件）或设备树文件，通过网络（scp）或 U 盘等方式拷贝到开发板的/userdata/等可写目录。

· 测试：在开发板上加载新的模块进行验证，无需反复烧录整个系统。